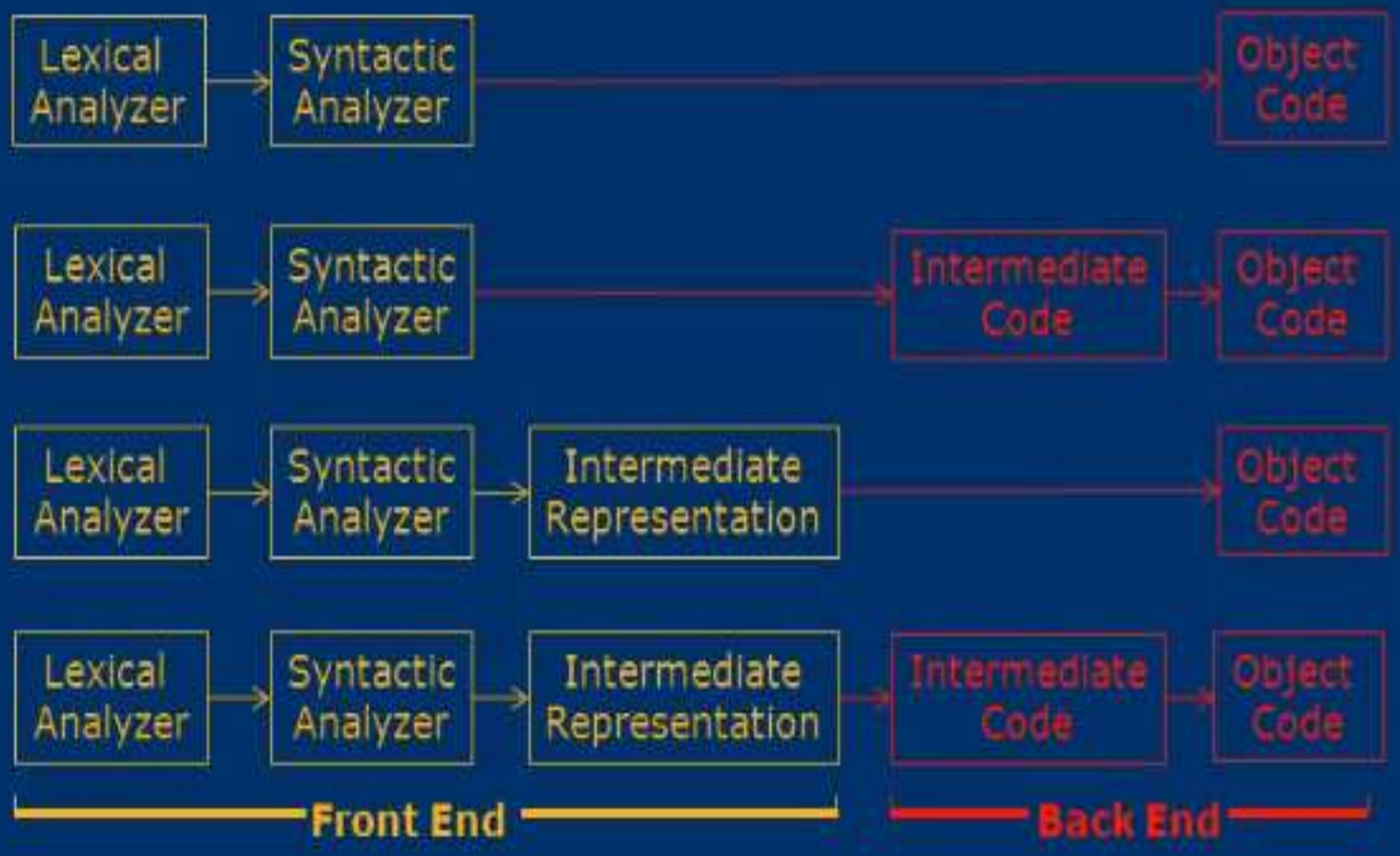
Introduction to Code Generation

- After syntactic analysis, we have a number of options to choose from:
  - generate object code directly from the parse
  - generate intermediate code, and then generate object code from it
  - generate an intermediate abstract representation, and then generate code directly from it
  - generate an intermediate abstract representation, generate intermediate code, and then the object code
- All these options have one thing in common: they are all based on syntactic information gathered in the semantic analysis

Introduction to Code Generation



Interm. Representations & Code

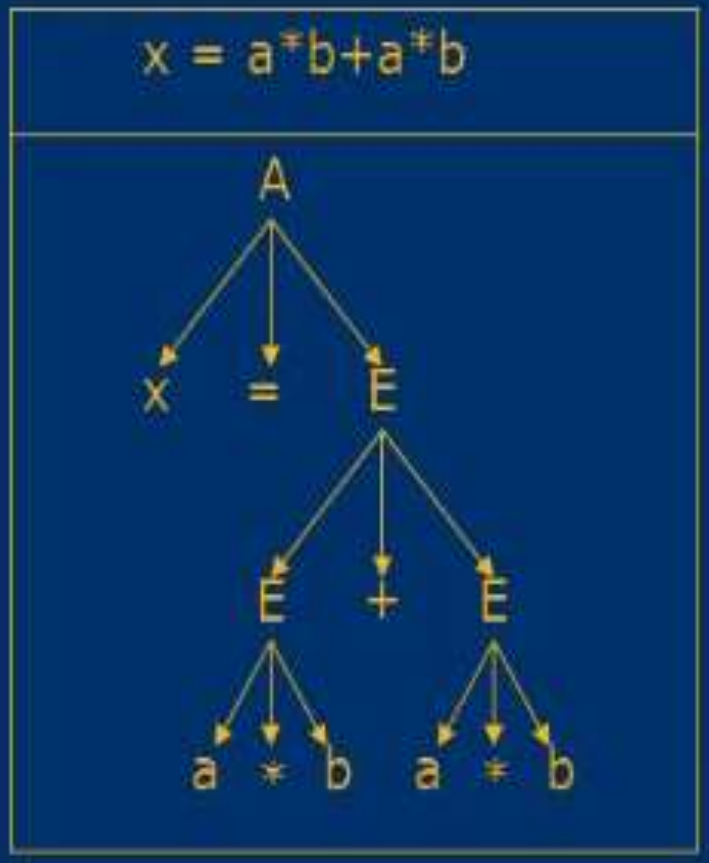
- **Intermediate representations** synthetize the syntactic information gathered during the parse, generally in the form of a tree or directed graph.
- Intermediate representations enable high-level code optimization.
- **Intermediate code** is a low-level coded (text) representation of the program, directly translatable to object code.
- Intermediate code enables low-level, architecture-dependent optimizations.

Abstract Syntax Trees

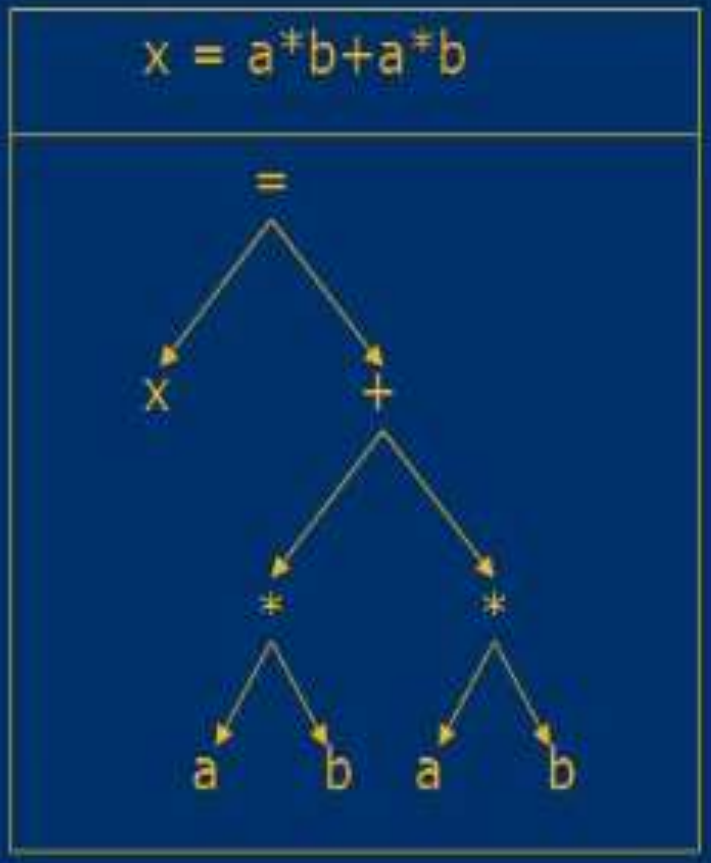
- Each node represents the application of a rule in the grammar
- A subtree is created only after the complete parsing of a right hand side
- Pointers to subtrees are sent up and grafted as upper subtrees are completed
- **Parse trees** (concrete syntax trees) emphasize the grammatical structure of the program
- **Abstract syntax trees** emphasize the actual computations to be performed. They do not refer to the actual non-terminals defined in the grammar, hence their name.

Parse vs Abstract Syntax Trees

Parse Tree



Abstract Syntax Tree

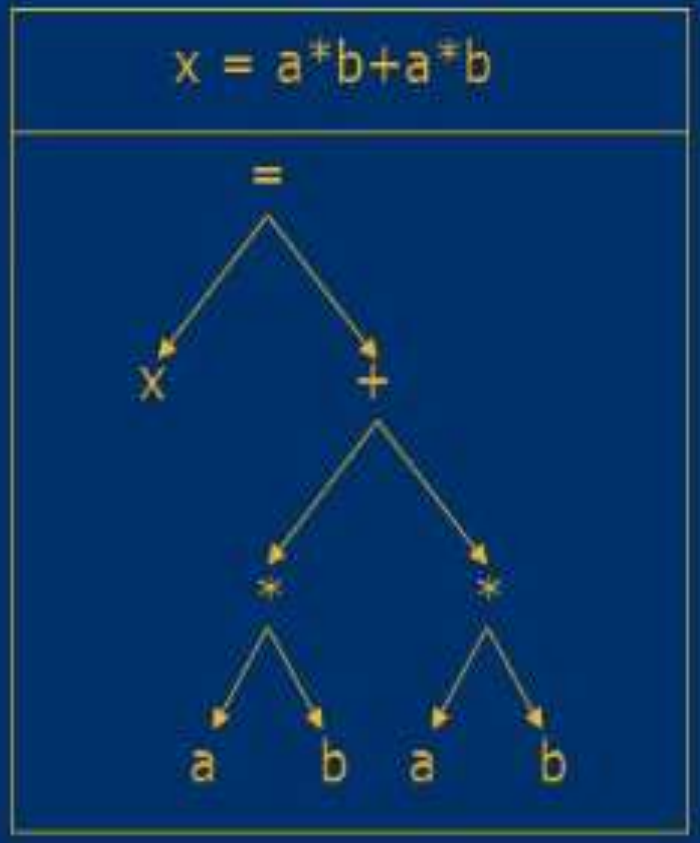


Directed Acyclic Graphs (DAG)

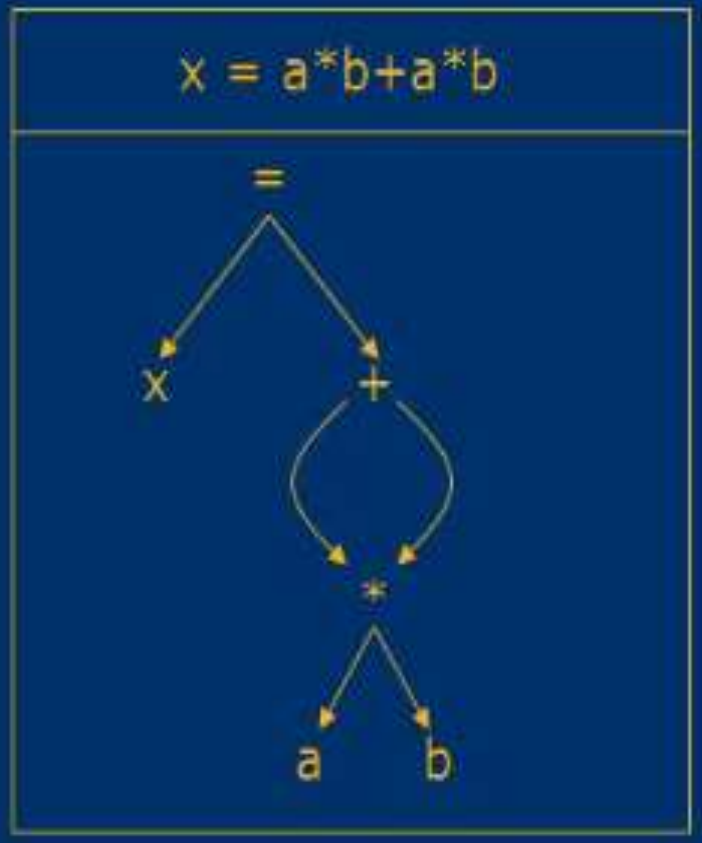
- **Directed acyclic graphs** (DAG) are a relative of syntax trees: they are used to show the syntactic structure of valid programs in the form of a "tree".
- In DAGs, the nodes for repeated variables and expressions are merged into a single node.
- DAGs are more complicated to build than syntax trees, but directly implement lots of code optimization by avoiding redundant operations.

AST vs DAG

Abstract Syntax Tree



Directed Acyclic Graph



Postfix Notation

- Every expression is rewritten with its operators at the end, e.g.:

a+b	⇒ ab+
a+b*c	⇒ abc*+
if A then B else C	⇒ ABC?
If A then if B then C else D else E	⇒ ABCD?E?
x=a*b+a*b	⇒ xab*ab*+=

- Easy to generate from a bottom-up parse
- Can be generated from a syntax tree using postorder traversal

Postfix Notation

- Its nature allows it to be naturally evaluated with the use of a stack
- Operands are pushed onto the stack; operators pop the right amount of operands from the stack, do the operation, then push the result back onto the stack.
- However, this notation is restricted to simple expressions such as in arithmetics where every rule conveys an operation
- It cannot be used for the expression of most programming languages constructs

Three-Address Code

- Three-address codes (3AC) is an intermediate language that maps directly to "assembly pseudo-code", i.e. architecture-dependent assembly code
- It breaks the program into short statements requiring no more than three variables and no more than one operator, e.g:

source	3AC
x = a+b*c	t := b*c x := a+t

Three-Address Code

- The temporary variables are generated at compile time and added to the symbol table
- In the generated code, the variables will refer to actual memory cells. Their address (or alias) is also stored in the symbol table
- 3AC can also be represented as quadruples, which are even more related to assembly languages

3AC	ASM
t := b*c	L 3,b M 3,c ST 3,t
x := a+t	L 3,a A 3,t ST 3,x

3AC	Quadruples
t := b*c	MULT t,b,c
x := a+t	ADD x,a,t

Intermediate Languages

- In this case, we generate code in a language for which we already have a compiler or interpreter
- Such languages are generally very low-level and dedicated to the compiler construction task
- It provides the compiler writer with a "virtual machine"
- Various compilers can be built using the same virtual machine
- The virtual machine compiler can be compiled on different machines to provide a translator to various architectures.